ASSIGNMENT - 1.5

Name: P. Sanjay Kumar
Roll No: 2403A52L06
Batch-50

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week1 - Friday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:1.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to*

		complete
1	Lab 1: Environment Setup – *GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow* **Lab Objectives:** ❖ To install and configure GitHub Copilot in Visual Studio Code. ❖ To explore AI-assisted code generation using GitHub Copilot. ❖ To analyze the accuracy and effectiveness of Copilot's code suggestions. ❖ To understand prompt-based programming using comments and code context **Lab Outcomes (LOs):** After completing this lab, students will be able to: ❖ Set up GitHub Copilot in VS Code successfully. ❖ Use inline comments and context to generate code with Copilot. ❖ Evaluate AI-generated code for correctness and readability. ❖ Compare code suggestions based on different prompts and programming styles.	Week1 - Monday
	Task 0 ❖ Install and configure GitHub Copilot in VS Code. Take screenshots of each step. Expected Output ❖ Install and configure GitHub Copilot in VS Code. Take screenshots of each step.	
	Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions) ❖ **Scenario** You are developing a **basic text-processing utility** for a messaging application. ❖ **Task Description** Use GitHub Copilot to generate a Python program that: ➢ Reverses a given string ➢ Accepts user input ➢ Implements the logic directly in the main code ➢ Does not use any user-defined functions	

❖ **Expected Output**

- Correct reversed string
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs

Prompt:
write a python code to reverse a string without user defined functions input should be taken from the user
Code:
Code:

Welcome × Lab Assignment 1.5.py ●

Lab Assignment 1.5.py > ...

```python
string = input("Enter a string: ")
reversed_string = string[::-1]
print(f"Original string: {string}")
print(f"Reversed string: {reversed_string}")
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMIN

```
PS C:\Users\sanjay\AI Assistant coding>
5.py"
Enter a string: sanjay
Original string: sanjay
Reversed string: yajnas
PS C:\Users\sanjay\AI Assistant coding>
```

JUSTIFICATION:
Memory Efficiency: It creates a new string in a single, highly optimized pass.
Readability: For experienced Python developers, this is the "standard" way. However, for beginners, the [::-1] syntax can look a bit like "magic" code.

Task 2: Efficiency & Logic Optimization (Readability Improvement)

❖ **Scenario**
The code will be reviewed by other developers.

❖ **Task Description**
Examine the Copilot-generated code from **Task 1** and improve it by:

- Removing unnecessary variables
- Simplifying loop or indexing logic

- Improving readability
- Use Copilot prompts like:
  - *"Simplify this string reversal code"*
  - *"Improve readability and efficiency"*

Hint:
Prompt Copilot with phrases like
"optimize this code", *"simplify logic"*, or *"make it more readable"*

❖ **Expected Output**
- Original and optimized code versions
- Explanation of how the improvements reduce time

Prompt:
Make The Code more readable and decrease the time complexityMake The Code more readable and decrease the time complexity

Code:

Lab Assignment 1.5.py > user_input

```python
user_input = input("Enter a string: ")
reversed_str = user_input[::-1]
print(f"Original:  {user_input}")
print(f"Reversed:  {reversed_str}")
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORT

```
PS C:\Users\sanjay\AI Assistant coding> & C:\Use
5.py"
Enter a string: sanjay
Original:  sanjay
Reversed:  yajnas
PS C:\Users\sanjay\AI Assistant coding>
```

JUSTIFICATION:
Readability: It is very explicit. A human reading the code knows exactly what is happening: you are reversing the string and then joining the characters back together. Overhead: Technically, this is slower than slicing. Here is why: 1. reversed(string) creates an iterator object. 2. .join() then has to loop through that iterator and build the string. These extra steps (creating an object and calling a method) add "overhead" that makes it slower than the direct slicing method.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

❖ **Scenario**

The string reversal logic is needed in **multiple parts** of an application.

❖ **Task Description**

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to reverse a string
- Returns the reversed string
- Includes meaningful comments (AI-assisted)

❖ **Expected Output**

- Correct function-based implementation
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs

PROMPT:

write a python code that reverse a string with user defined functions that return a reversed string and comments for understand the code.

Code:

```python
def reverse_string(s):
    # This function takes a string as input and returns the reversed string
    return s[::-1]

# Take input from the user
user_input = input("Enter a string: ")

# Call the function and store the result
result = reverse_string(user_input)

# Print the reversed string
print("Reversed string:", result)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\sanjay\AI Assistant coding> & C:\Users\
oding/Lab Assignment 1.5.py"
Enter a string: hello
Reversed string: olleh
PS C:\Users\sanjay\AI Assistant coding>
```

JUSTIFICATION:

Reusability: By defining a function, you can reverse strings multiple times throughout a larger program without rewriting the slicing logic. Modularity: It separates the core logic (reversing) from the user interface (input/output), making the code cleaner and easier to maintain.

	Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions) ❖ **Scenario** You are asked to justify design choices during a code review. ❖ **Task Description** Compare the Copilot-generated programs: ➢ Without functions (Task 1) ➢ With functions (Task 3) Analyze them based on: ➢ Code clarity ➢ Reusability ➢ Debugging ease ➢ Suitability for large-scale applications ❖ **Expected Output** Comparison table or short analytical report	
	Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal) ❖ **Scenario** Your mentor wants to evaluate how AI handles **alternative logic paths**. ❖ **Task Description** Prompt GitHub Copilot to generate: ➢ A **loop-based** string reversal approach ➢ A **built-in / slicing-based** string reversal approach ❖ **Expected Output** ➢ Two correct implementations ➢ Comparison discussing: ▪ Execution flow ▪ Time complexity ▪ Performance for large inputs ▪ When each approach is appropriate **Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.**	